Dr. JAEHYUK LEE, Ph.D.

470-861-6020 https://ruach.github.io jaehyukl@nvidia.com

RESEARCH INTEREST

> *I am interested in computer systems security, especially in: hardware/software codesign for security, Trusted Execution Environment (TEE), secure IO design for TEE, software/hardware vulnerability research, side-channel attacks and mitigations, and fuzzing*

TECHNICAL SKILLS

Software: C, C++, Rust, Scala, Python, TF-A, TF-RMM, Caliptra FW, PSC FW (NVIDIA IROT), Linux Kernel, KVM, QEMU, ARM/X86 Assembly, Fuzzing, GDB, CoCLI

Hardware: GEM5 (x86 simulator), Processor Pipeline, Cache, TLB, Microcode, HW Side Channel, (System)Verilog, Chisel (RISC-V), Intel SGX & TDX, ARM CCA & TrustZone, IOMMU, PCIe

Spec: Entity Attestation Token (EAT), Concise Reference Integrity Manifest (CoRIM), DICE Protection Environment (DPE), Security Protocol and Data Model (SPDM), TEE Device Interface Security Protocol (TDISP)

WORK EXPERIENCE

Senior Security Engineer: NVIDIA Corporation, Santa Clara, CA **Jul 2024 – Current**

- Owned device evidence and CoRIM generation for Vera and Grace confidential computing
  - Implemented cross-layer firmware support across *RMM, ATF, PSCFW (IROT), and Caliptra DPE* to enable end-to-end *Vera CCA* evidence generation.
  - Implemented the software stack to enable *Baremetal Secure AI* attestation evidence generation for Vera and Grace CPUs.
  - Developed CoRIM for Vera CCA and Vera/Grace baremetal attestation.
- Owned Live Firmware Activation (LFA) support for RMM
  - Built the supporting software stack across *RMM, ATF, and Vera IROT* to enable live firmware activation.
- Experienced in the end-to-end SBIOS firmware development lifecycle, including build, validation, testing, and firmware package releases.
- Engineered secure tenant data loading (e.g., ML models) for NVFlare by integrating encrypted disk and early attestation key provisioning directly into initramfs, securing the boot sequence.

Postdoc Researcher: Georgia Institute of Technology, Atlanta, GA **Jan 2022 – Jul 2024**

- Developed an automated fuzzer selection tool that dynamically selects fuzzer(s) based on runtime analysis [4]. Anyone having no knowledge of fuzzers can simply utilize it for automate vulnerability discovery
- Experienced in vulnerability research on Intel and ARM's latest hardware feature designed for confidential VM (TDX & CCA). Identified new side-channel issues related to resource allocation in the construction of confidential VMs
- Implemented a secure IO path between confidential VMs and peripherals such as GPUs by leveraging isolation guarantees of IOMMU (SMMU) and ARM CCA, avoiding encryption overhead [1]

Postdoc Researcher: Korea Advanced Institute of Science and Technology, Korea **Mar 2021 – Dec 2021**

- Implemented cache side-channel attack on X86 and Apple M1 chips, unveiling the memory access pattern without evicting the victim's cache entries and *completely bypassing side-channel defenses* monitoring cache evictions [10]
- Took the lead in managing and mentoring a group of 20 Ph.D. students

Research Intern: Georgia Institute of Technology, Atlanta, GA **Nov 2019 – May 2020**

- Designed and implemented a *hardware-based side-channel mitigation* on the GEM5 simulator, enabling user-space applications to subscribe to micro-architectural events (cache and TLB evictions) [3]

Research Intern: Microsoft Research, Redmond, WA **May 2016 – August 2016**

- Researched memory corruption vulnerabilities in Intel SGX and executed a practical ROP attack against encrypted binaries, demonstrating compromise of confidentiality guarantees [5].
- Developed a secure CDN that enables enterprises to offload sensitive user data to untrusted third-party cloud providers without compromising confidentiality.

PUBLICATION & PATENT (826 citations)

top-tier conferences (SP, USENIX Security, NDSS) and Journal (TDSC)

[1] **PORTAL: Fast and Secure Device Access with Arm CCA for Modern Arm Mobile SoC** | Paper **SP 2025**

- Eliminated per-transfer encryption overhead for Arm CCA Realm VMs accessing DMA-capable peripherals (GPU, NPU) by migrating SMMU stage-2 page table ownership into TF-RMM and introducing new RMI calls (`RMI_DEVICE_ATTACH/DETACH`) between KVM and RMM, enabling hardware-enforced isolated DMA regions without cryptographic cost.
- Extended the Linux SMMU driver to delegate page table management to RMM; implemented the Realm-side driver and the full RMI call path across KVM, TF-RMM, and the SMMU hardware controller.

[2] **Survey On DeFi Users' Perception of Security Breaches and Countermeasures** | Paper **USENIX 2024**

- Revealed that DeFi victims overwhelmingly skip post-incident security review and instead seek new platforms to recover losses quickly — behavior analogous to gambling addiction — suggesting industry regulation is more effective than user education alone.

- Conducted a two-phase study (N=14 semi-structured interviews + N=493 survey); performed qualitative coding of interview transcripts using Dedoose, mapping responses to a taxonomy of DeFi attack types (rug pulls, flash loans, oracle manipulation, access-control bugs).

[3] **SENSE: Enhancing Microarchitectural Awareness via Subscription-Based Notification** | Paper **NDSS 2024**
- Addressed the lack of microarchitectural visibility in TEEs that leaves them blind to cache/TLB side-channel attacks by designing a subscription-based ISA extension that delivers hardware events directly to the enclave handler — without requiring OS cooperation.
- Extended the GEM5 out-of-order CPU pipeline with new micro-ops and added event-tracking metadata bits to the cache and TLB models; implemented the enclave-side handler dispatch path and evaluated detection latency against PRIME+PROBE attacks.

[4] **autofz: Automated Fuzzer Composition at Runtime** | Paper | Hacker News **USENIX 2023**
- Eliminated the need for per-target fuzzer selection by building a runtime coordinator that measures each fuzzer's coverage gain via AFL bitmaps and reallocates CPU time with AIMD, lifting bug-finding throughput by up to 3× over the best single fuzzer across 12 targets.
- Implemented the orchestration layer in Python using Linux cgroups for CPU pinning; integrated 12 fuzzers (AFL, AFLFast, MOpt, RedQueen, QSYM, Angora, and others) under a unified coverage-feedback interface.

[5] **Hacking in Darkness: Return-oriented Programming against Secure Enclaves** | Paper | Talk | Demo **USENIX 2017**
- Demonstrated the first blind ROP attack against Intel SGX: exploited the deterministic AEX fault oracle to probe enclave memory word-by-word and reconstruct a gadget map — without any binary knowledge — then chained a `memcpy()` gadget into a full key-exfiltration payload.
- Built the AEX-based memory scanner and ROP-chain generator in C/x86 assembly; reverse-engineered EPCM/TCS/SSA layouts to locate gadgets and craft the exploit entirely from outside the enclave.

[6] **SGX-Bomb: Locking Down the Processor via Rowhammer Attack** | Paper | Hacker News | Demo **SysTEX 2017**
- Showed that an enclave can weaponize Rowhammer to permanently lock the Memory Encryption Engine (MEE) and force a machine reboot — rendering SGX's integrity guarantees a denial-of-service vector for cloud multi-tenant scenarios.
- Wrote a kernel driver to reverse-engineer DRAM bank/row mapping from physical addresses; implemented the double-sided Rowhammer loop in x86 assembly inside an enclave to flip MEE metadata bits and trigger a machine lockdown.

[7] **PrivateZone: Providing a Private Execution Environment using ARM TrustZone** | Paper **TDSC 2016**
- Extended ARM TrustZone to protect arbitrary user-space processes without porting them to the Secure World, by intercepting Linux scheduling events in Monitor mode and switching per-process stage-2 page tables on each context switch.
- Implemented the stage-2 page table allocator and the Monitor-mode context-switch handler in ARM assembly; modified the Linux scheduler to emit SVC calls that trigger the switch between trusted and untrusted page tables.

[8] **OpenSGX: An Open Platform for SGX Research** | Paper | Wikipedia: SGX **NDSS 2015**
- Lowered the barrier to SGX research before hardware was available by building an instruction-level emulator in QEMU that faithfully reproduced EPCM, SECS, TCS, and SSA data structures, enabling enclave development and testing on commodity machines.
- Implemented `EENTER`/`EEXIT` context-switch stubs in x86 assembly, EPCM management in QEMU's softmmu, and a `sgxlib` user library with a stub-call layer that routes enclave system calls through an untrusted trampoline.

[9] **A Rule Extraction Method Using Relevance Factor for FMM Neural Networks** | Paper **KTSDE 2013**
- Enhanced Fuzzy Min-Max (FMM) neural network by incorporating the frequency of features in training.
- Resolved ambiguity by prioritizing clarity and relevance, while ensuring distinctiveness between features and patterns.

[10] **PRIME+RETOUCH: When Cache is Locked and Leaked** | Paper **Arxiv**
- Identified a blind spot in eviction-based cache side-channel defenses: Tree-PLRU replacement metadata can be manipulated to "retouch" a cache set so the victim line survives eviction silently — bypassing Intel TSX-based monitors and Apple M1 PMU counters.
- Reverse-engineered the Tree-PLRU state machine on recent Intel and Apple M1 caches; built the PRIME+RETOUCH PoC in x86/ARM assembly using pointer-chasing and memory barriers; demonstrated cross-core and cross-process secret recovery.

[11] **Apparatus and method for open and private IoT gateway using Intel SGX** | Patent **KR102162018B1**
- The invention introduces a secure IoT gateway design, utilizing Intel SGX for protocol conversion and encrypted data transfer. It includes enclave designs for cross-platform communication, encrypted key management, and secure updates.

SOFTWARE ARTIFACTS

Autofz:	https://github.com/sslab-gatech/autofz	⑂13	★85
OpenSGX:	https://github.com/sslab-gatech/opensgx	⑂82	★308
SGXBomb:	https://github.com/sslab-gatech/sgx-bomb	⑂4	★18

EDUCATION

Institution	Degree	Field of Study	Dates	GPA
Korea Advanced Institute of Science and Technology	*Ph.D.*	Information Security	Aug 2015 - Feb 2021	3.95
Korea Advanced Institute of Science and Technology	*M.S.*	Information Security	Sep 2013 - Aug 2015	4.1
Handong Global University	*B.S.*	Computer Science	Mar 2010 - Aug 2013	4.2

PROFESSIONAL ACTIVITIES

[1] **Journal Review**

- IEEE Transactions on Dependable and Secure Computing — Impact Factor: 7.3, *2024*
- Computer & Security — Impact Factor: 5.7, *2024*
- Journal of Information Security and Applications — Impact Factor: 5.6, *2024*
- IEEE Access — Impact Factor: 3.9, *2024*
- ACM Transactions on Internet Technology — Impact Factor: 3.9, *2024*
- IEEE Journal of Communications and Networks — Impact Factor: 3.6, *2024*

[2] **Conference Review**

- IEEE Mobile Ad Hoc and Smart Systems (MASS) — *2024*
- IEEE International Conference on Parallel and Distributed Systems (ICPADS) — *2024*

[3] **External Reviewer**

- IEEE Symposium on Security and Privacy (Oakland) — *2016, 2017, 2018, 2020, 2021, 2022*
- USENIX Security Symposium (Security) — *2015, 2021, 2022, 2023, 2024*
- ACM Conference on Computer and Communications Security (CCS) — *2015, 2017, 2018, 2019, 2023*
- Network and Distributed System Security Symposium (NDSS) — *2019, 2020, 2021*
- ACM ASIA Conference on Computer and Communications Security (ASIACCS) — *2015, 2016, 2018*
- ACM Symposium on Operating Systems Principles (SOSP) — *2021*
- ACM International World Wide Web Conference (WWW) — *2018*

INVITED TALK

[1] **A Novel Cache Side Channel Attack without Attacker Eviction**

- Presented at Intel Side Channel Academic Program (SCAP), Virtual, Sep 2020

[2] **Hacking in Darkness: Return-oriented Programming against Secure Enclaves**

- Presented at the 26th Usenix Security | Presentation Link, Vancouver, BC, Canada, Aug 2017
- Presented at KimchiCon | Presentation Link, Daejeon, South Korea, Aug 2017

[3] **Tutorial: Trusted Execution Environment: TrustZone, ITP and SGX**

- Korea Institute of Information Security and Cryptology, Seoul, South Korea, Apr 2015

THESIS

Unintended Consequences of System Design: Unpremeditated Usage of Benign System Components Devastates Security	Ph.D. Thesis.
Detecting Emulated Environment: Exploiting Behavioral Discrepancies In QEMU	M.S. Thesis.